

Day 51 – Hibernate Day 09

One-to-Many & Many-to-One Mapping

What is One-to-Many Mapping?

One-to-Many Mapping is used when one record of one table is connected with multiple records of another table.

In simple words:

One parent object can have many child objects.

But every child object belongs to only one parent.

Real Example

Think about a classroom.

One Teacher teaches many Students.

One Student belongs to one Teacher.

In our project:

One Question has multiple Answers.

Every Answer belongs to only one Question.

Relationship:

Question Entity (Parent)

```
package com.nit.Mapping2;

import java.util.List;
import jakarta.persistence.*;
import lombok.Data;
import lombok.ToString;

@Entity
@Table(name="que_value")
@Data
public class Question {

    @Id
```

```

    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="que_id")
    private int id;

    private String questions;

    @OneToMany
    @ToString.Exclude
    private List<Answers> answer;
}

```

Explanation

- `@Entity` → Makes this class a Hibernate entity.
- `@Table(name="que_value")` → Creates the que_value table.
- id → Primary key.
- answer → Stores multiple Answer objects.

Here,

```
List<Answers> answer;
```

means one question can contain many answers.

Answers Entity (Child)

```

package com.nit.Mapping2;

import jakarta.persistence.*;
import lombok.Data;
import lombok.ToString;

@Entity
@Table(name="ans_value")
@Data
public class Answers {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    @Column(name="ans_id")
    private int id;
}

```

```

private String answers;

@ManyToOne
@JoinColumn(name="que_id")
@ToString.Exclude
private Question question;
}

```

Explanation

Every answer keeps a reference to its parent question.

```
private Question question;
```

means each answer belongs to only one question.

Main Program

```

public class McqMultipleCorrect {

    public static void main(String[] args) {

        Question question = new Question();
        question.setQuestions("What is Hibernate");

        Answers answer1 = new Answers();
        answer1.setAnswers("Hibernate is an ORM");
        answer1.setQuestion(question);

        Answers answer2 = new Answers();
        answer2.setAnswers("Hibernate follows Jpa Rules");
        answer2.setQuestion(question);

        Answers answer3 = new Answers();
        answer3.setAnswers("Hibernate is a Framework");
        answer3.setQuestion(question);

        List<Answers> data = new ArrayList<>();
        data.add(answer1);
    }
}

```

```

        data.add(answer2);
        data.add(answer3);

        question.setAnswer(data);

        Session currentSession =
            ConnectionEstablisher.getConnection().openSession();

        Transaction transaction =
            currentSession.beginTransaction();

        currentSession.persist(question);
        currentSession.persist(answer1);
        currentSession.persist(answer2);
        currentSession.persist(answer3);

        System.out.println("Fetching the data");

        Question reference =
            currentSession.getReference(Question.class,1);

        System.out.println(reference.getQuetions());

        reference.getAnswer()
            .stream()
            .forEach(System.out::println);

        transaction.commit();
        currentSession.close();
    }
}

```

Understanding @OneToMany

```

@OneToMany
private List<Answers> answer;

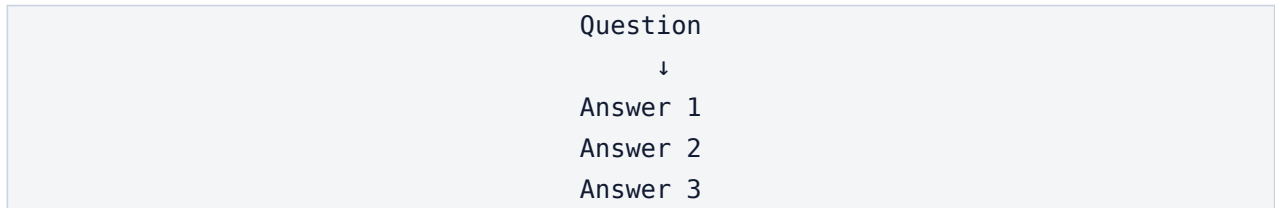
```

This annotation creates a relationship where one object stores multiple objects.

Why do we use List<Answers>?

Because one question has multiple answers.

Example:



A single object cannot store multiple answers, so we use a List.

Where do we write @OneToMany?

Write it inside the parent class.

Example:

```
public class Question {  
  
    @OneToMany  
    private List<Answers> answer;  
}
```

The parent stores the collection.

When do we use it?

Use it when:

- One object owns multiple objects.
- Parent should access all child records.

Examples:

- Customer → Orders
- Department → Employees
- Question → Answers

How do we write it?

Syntax:

```
@OneToMany  
private List<Child> children;
```

Here,

```
private List<Answers> answer;
```

stores all Answer objects.

What happens if we don't write @OneToMany?

Hibernate treats List<Answers> like a normal Java list.

Problems:

- Relationship won't be created.
- Hibernate won't know these answers belong to the question.
- Child records cannot be fetched automatically.

Understanding @ManyToOne

```
@ManyToOne
@JoinColumn(name="que_id")
private Question question;
```

This annotation tells Hibernate:

"Many Answer records belong to one Question."

Every Answer stores the parent Question.

Relationship:

Where do we write @ManyToOne?

Write it inside the child class.

Example:

```
public class Answers {

    @ManyToOne
    private Question question;
}
```

Every child stores one parent.

When do we use it?

Use it when many child records belong to one parent.

Examples:

- Many Employees → One Department

- Many Orders → One Customer
- Many Answers → One Question

Why is @JoinColumn(name="que_id") used?

```
@JoinColumn(name="que_id")
```

It creates the foreign key column.

Instead of Hibernate generating its own name, we create

```
que_id
```

which clearly connects both tables.

What happens if we don't write @ManyToOne?

Then every Answer becomes an independent record.

Problems:

- No foreign key.
- No parent-child relationship.
- Question cannot identify its answers.

Understanding @ToString.Exclude

```
@ToString.Exclude
```

This annotation comes from Lombok.

It removes a field from the automatically generated toString() method.

Why do we use it?

Both classes refer to each other.

```
Question
  ↓
Answers
  ↓
Question
  ↓
Answers
  ↓
...
```

If Lombok prints both objects continuously, it creates an infinite loop.

@ToString.Exclude stops that loop.

Where do we write it?

Write it on relationship fields.

Example:

```
@ToString.Exclude
private List<Answers> answer;
```

and

```
@ToString.Exclude
private Question question;
```

What happens if we don't write it?

When printing

```
System.out.println(question);
```

Lombok tries to print:

```
Question
  Answers
    Question
      Answers
        Question
          ...
```

This causes

```
StackOverflowError
```

because printing never ends.

Data Inserted

Question	Answer
What is Hibernate	Hibernate is an ORM
What is Hibernate	Hibernate follows Jpa Rules
What is Hibernate	Hibernate is a Framework

Fetch Output

```
Fetching the data
```


Que : What is Hibernate

Ans is :

Answers(id=1, answers=Hibernate is an ORM)

Answers(id=2, answers=Hibernate follows Jpa Rules)

Answers(id=3, answers=Hibernate is a Framework)

Database Tables Created

Table: que_value

QUE_ID	QUESTIONS
1	What is Hibernate

Table: ans_value

ANS_ID	ANSWERS	QUE_ID (FK)
1	Hibernate is an ORM	1
2	Hibernate follows Jpa Rules	1
3	Hibernate is a Framework	1

Here,

QUE_ID is the Foreign Key, which connects every Answer with its parent Question.

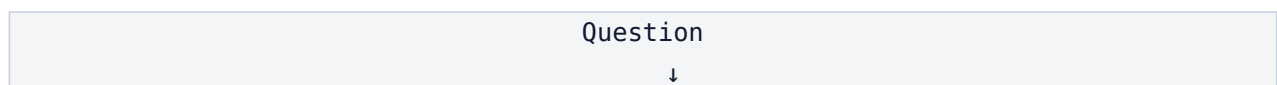
How Hibernate Stores the Relationship

Notice that all three Answer records store the same QUE_ID, which links them to the single Question.

Execution Flow (Step by Step)

This is how the program runs internally.

Step 1: Create one Question object.



Step 2: Create three Answer objects.

```
Answer1  
Answer2  
Answer3
```

Step 3: Connect every Answer with the Question.

```
answer1.setQuestion(question);  
answer2.setQuestion(question);  
answer3.setQuestion(question);
```

Now every Answer knows its parent.

Step 4: Store all Answers inside a List.

```
List<Answers> data = new ArrayList<>();
```

Step 5: Attach the List to the Question.

```
question.setAnswer(data);
```

Now the Question knows all its Answers.

Step 6: Open Hibernate Session.

```
openSession();
```

Database connection starts.

Step 7: Begin Transaction.

```
beginTransaction();
```

Hibernate is ready to save data.

Step 8: Save Question.

```
persist(question);
```

Parent record is inserted.

Step 9: Save every Answer.

```
persist(answer1);  
persist(answer2);  
persist(answer3);
```

Child records are inserted with QUE_ID.

Step 10: Fetch Question.

```
Question reference =  
    currentSession.getReference(Question.class,1);
```

Hibernate gets Question ID 1.

Step 11: Fetch related Answers.

```
reference.getAnswer();
```

Hibernate automatically loads all child records.

Step 12: Print Answers.

```
answer.stream().forEach(System.out::println);
```

All three Answer objects are displayed.

Step 13: Commit Transaction.

```
transaction.commit();
```

Changes are permanently saved.

Step 14: Close Session.

```
currentSession.close();
```

Database resources are released.

Quick Revision

Annotation	Purpose
@OneToMany	One parent stores multiple child objects.
@ManyToOne	Many child objects belong to one parent.
@JoinColumn	Creates the foreign key column.
@ToString.Exclude	Prevents infinite recursion while printing objects.

Remember:

- Question is the Parent.
- Answers is the Child.
- @OneToMany is written in the parent class.
- @ManyToOne is written in the child class.
- @JoinColumn creates the foreign key (QUE_ID).
- @ToString.Exclude prevents StackOverflowError caused by continuous object printing.